Cahier des Charges — Projet de Développement Logiciel (HAX712X)

PyArena : Simulateur Stochastique, Génie Logiciel et Dashboard Décisionnel

Bensaid Bilel

2026-09-13

Table des matières

Présentation Générale et Organisation **2**
Objectifs Pédagogiques . . . 2
Modalités de Constitution des Équipes (Promotion : 32 étudiants) . . . 3
Matrice Contractuelle d'Exigences : Groupes de 3 vs Groupes de 4 . . . 3

Modèle Mathématique et Règles Métier du Combat **3**
Caractéristiques et Normalisation des Combattants . . . 4
Le Système d'Affinités Élémentaires . . . 4
Le Modèle Simplifié : Le Tournoi à 5 Types (Groupes de 3) . . . 4
Le Modèle Complet : 18 Types et Doubles Types (Groupes de 4 / Bonus) . . . 5
Déroulement Déterministe et Stochastique d'un Tour . . . 5
1. Phase d'Initiative . . . 5
2. Phase de Frappe de l'Attaquant . . . 5
3. Phase de Résolution et Riposte . . . 6

Architecture Logicielle et Standards de Développement **6**
Arborescence Normée du Dépôt Git . . . 6
Règles d'Or de la Programmation Orientée Objet (POO) . . . 7
Packaging et Intégration Continue (GitHub Actions) . . . 7

Pipeline de Données, Cache Local et Simulation **8**
Ingestion PokéAPI et Cache Hors-Ligne . . . 8
Moteur de Simulation Monte-Carlo Parallélisé . . . 8

Spécifications du Tableau de Bord Streamlit **9**
Module 1 : Simulateur de Duel et Télémétrie en Direct (Commun) . . . 9
Module 2 : Catalogue et Analyse Exploratoire des Données (Commun) . . . 9
Module 3 : Cartographie Géospatiale des Arènes d'Occitanie (Commun) . . . 9
Module 4 : Analyseur de Synergie d'Équipe (Obligatoire Groupes de 4 / Bonus Groupes de 3) . . . 9

Spécifications Exhaustives des 10 Extensions Thématiques **10**
Cadre Technique Commun et Équité . . . 10
Extension 1 : Objets Passifs et Équipements Stratégiques . . . 10

Extension 2 : Combats d'Équipes 3 contre 3 avec Relais (Tag-Team) 11
Extension 3 : Matrice Complète de Méta-Jeu et Détection d'Intransitivité 13
Extension 4 : Choix Stratégique d'Attaques (Précision vs Puissance) 13
Extension 5 : Système Stochastique d'Altérations d'État 14
Extension 6 : Gestion des Points de Puissance (PP) et Action Lutte 15
Extension 7 : Accélération Vectorisée et Profilage Numérique (NumPy / Numba) 16
Extension 8 : Modélisation Paramétrique de Niveaux et Détermination de Seuil Critique . 17
Extension 9 : La Route des Champions (Occitanie — TSP sous Contraintes de Niveau) . 18
Extension 10 : Dynamique Météorologique et Modificateurs d'Arène (Climats) 20

Politique de Valorisation et Points de Bonus **21**
Barème indicatif des initiatives possibles : 21

Travail Collaboratif, Traçabilité Git et Intégrité **21**
Standards Git Obligatoires 21
Détection du Plagiat et Contrôle d'Activité 22

Modalités d'Évaluation et Grilles de Notation **22**
Note 1 : Code, Architecture et Produit (/20 — Mixte, Coeff 3) 22
A. Socle Produit Collectif (/12 points — Note d'Équipe) 23
B. Contribution Technique Individuelle (/8 points — Note Individuelle) 23
Note 2 : Soutenance Orale et Maîtrise Individuelle (/20 — Mixte, Coeff 2) 24

Calendrier Prévisionnel et Jalons de Suivi **25**

Présentation Générale et Organisation

Le projet de l'UE **Développement Logiciel (HAX712X)** constitue le fil conducteur pratique du semestre pour la promotion de **Master 1 Sciences des Données (SSD)**.

L'objectif est de concevoir, développer, tester unitairement, documenter et déployer une bibliothèque logicielle modulaire en Python simulant une arène de duels au tour par tour (**moteur headless**), articulée avec un pipeline d'ingestion de données web, un moteur de simulation stochastique parallélisé et un **tableau de bord interactif d'aide à la décision (Streamlit)**.

Objectifs Pédagogiques

1. **Génie logiciel & Qualité de code :** Programmation orientée objet stricte, encapsulation, modularité, hiérarchie d'exceptions typées, packaging standard (`pyproject.toml`) et documentation.
2. **Industrialisation & Intégration Continue (CI) :** Écriture de tests unitaires automatisés (`pytest`), mesure continue de la couverture de code (`pytest-cov`) et validation des jalons par des workflows GitHub Actions.
3. **Data Science & Calcul Parallèle :** Modélisation stochastique par simulations de Monte-Carlo, distribution multi-cœurs (`multiprocessing` / `joblib`), ingestion d'API REST et mise en cache sur disque.
4. **Visualisation & Outils Décisionnels :** Visualisations interactives (Plotly), cartographie spatiale régionale (Folium) et ergonomie d'application web décisionnelle (Streamlit).

Modalités de Constitution des Équipes (Promotion : 32 étudiants)

Par contrainte d'effectif global (32 inscrits), la promotion est obligatoirement structurée en :

$$\textbf{32} \text{ étudiants} = (8 \times 3) + (2 \times 4) \implies \textbf{10} \text{ équipes au total}$$

Cette configuration garantit une bijection parfaite avec le catalogue des **10 extensions thématiques exclusives** : chaque groupe de la promotion développera une extension distincte, sans aucun doublon.

Matrice Contractuelle d'Exigences : Groupes de 3 vs Groupes de 4

Afin de préserver une équité absolue dans l'évaluation face au volume de travail disponible, le tableau suivant consigne les exigences minimales fermes selon la taille de l'équipe :

Dimension d'Évaluation	Équipe de 3 étudiants (Le Standard)	Équipe de 4 étudiants (Périmètre Renforcé)
Catalogue de données	1 Génération (id $\in [\![1, 151]\!]$)	**Générations 1 et 2 complètes** (id $\in [\![1, 251]\!]$)
Système des types	**Tournoi Élémentaire à 5 types** (matrice fixe fournie)	**18 types complets & Doubles types** (ingestion API)
Couverture `pytest`	Seuil minimal ferme de $\geq 70\%$	Seuil minimal renforcé de $\geq 85\%$
Modules Streamlit	3 modules (Duel, Catalogue/EDA, Carte Occitanie)	**4 modules** (+ Module Analyseur de Synergie d'Équipe)
Extension Métier	1 extension attribuée parmi les 10	1 extension attribuée parmi les 10
Audit Git Individuel	≥ 2 Pull Requests par membre	≥ 2 PRs créées **ET** ≥ 2 PRs revues (*Approval*) par membre

> **!** Égalité de Traitement sur les Extensions
>
> Les règles algorithmiques de chaque extension thématique sont **strictement identiques** pour un groupe de 3 ou un groupe de 4. La différence réside exclusivement dans le périmètre sur lequel l'extension s'exécute (151 créatures et 5 types pour les trinômes contre 251 créatures, 18 types et doubles types pour les quatuors).

Modèle Mathématique et Règles Métier du Combat

Le système dynamique modélisé est un duel discret au tour par tour opposant deux combattants. Le modèle est rigoureusement formel et autosuffisant : **aucune connaissance préalable de jeux vidéo n'est requise**.

Caractéristiques et Normalisation des Combattants

Chaque combattant C est modélisé par un vecteur d'état à huit composantes :

$$C = \langle \text{id}, \text{nom}, T_1, T_2, \text{PV}_{\max}, \text{PV}, A, D, V \rangle$$

- id $\in \mathbb{N}^*$: identifiant officiel PokéAPI ($[\![1, 151]\!]$ pour les trinômes ; $[\![1, 251]\!]$ pour les quatuors).
- nom $\in \mathbb{S}$: chaîne de caractères désignant le combattant (champ `name` de l'API).
- $T_1 \in \mathcal{T}$: type élémentaire principal, extrait de l'objet de la liste `types` dont la clé `slot` vaut 1.
- $T_2 \in \mathcal{T} \cup \{\emptyset\}$: second type élémentaire éventuel (`slot == 2`) ou $\emptyset$ si la créature est mono-type.
- Les statistiques brutes de base ($\text{Stat}_{\text{base}}$) proviennent de la liste `stats` du JSON API :
  - PV_{base} correspond à l'entrée `stat.name == "hp"`
  - A_{base} correspond à l'entrée `stat.name == "attack"`
  - D_{base} correspond à l'entrée `stat.name == "defense"`
  - V_{base} correspond à l'entrée `stat.name == "speed"`
- Ces caractéristiques sont normalisées au **Niveau 50 standard** par les formules d'échelonnement :

$$\text{PV}_{\max} = \text{PV}_{\text{base}} + 60$$
$$A = A_{\text{base}} + 5, \quad D = D_{\text{base}} + 5, \quad V = V_{\text{base}} + 5$$

- PV $\in [\![0, \text{PV}_{\max}]\!]$: points de vie courants du combattant.

Dès que PV $= 0$, l'entité est déclarée **K.-O.** (*fainted*) et cesse immédiatement toute action.

Le Système d'Affinités Élémentaires

Le Modèle Simplifié : Le Tournoi à 5 Types (Groupes de 3)

Pour les groupes de 3, l'arène restreint la participation aux créatures dont le type principal T_1 appartient à l'ensemble $\mathcal{T}_5 = \{\text{Normal}, \text{Feu}, \text{Eau}, \text{Plante}, \text{Électrik}\}$. Le coefficient multiplicateur $M(T_{\text{att}}, T_{\text{déf}}) \in \{0.5, 1.0, 2.0\}$ est lu directement dans la matrice fixe suivante :

Type Attaquant (T_{att}) / Type Défenseur ($T_{\text{déf}}$)	Normal	Feu	Eau	Plante	Électrik
Normal	1.0	1.0	1.0	1.0	1.0
Feu	1.0	0.5	0.5	2.0	1.0
Eau	1.0	2.0	0.5	0.5	1.0
Plante	1.0	0.5	2.0	0.5	1.0
Électrik	1.0	1.0	2.0	0.5	0.5

(Note d'implémentation : le client de données télécharge l'ensemble des 151 créatures sur disque, mais le sélecteur de l'arène filtre le catalogue pour ne retenir que les combattants dont $T_1 \in \mathcal{T}_5$, soit environ 80 créatures éligibles).

Le Modèle Complet : 18 Types et Doubles Types (Groupes de 4 / Bonus)

Pour les groupes de 4, le moteur ingère la table complète des 18 types via l'API (`https://pokeapi.co/api/v2/type/{id`

Lorsqu'une attaque de type T_{att} frappe un combattant possédant deux types défensifs $(T_{\text{déf},1}, T_{\text{déf},2})$, le multiplicateur global M_{total} est le **produit algébrique** des affinités élémentaires individuelles :

$$M_{\text{total}}(T_{\text{att}}, (T_{\text{déf},1}, T_{\text{déf},2})) = M(T_{\text{att}}, T_{\text{déf},1}) \times M(T_{\text{att}}, T_{\text{déf},2})$$

Ce produit croisé engendre quatre situations sans exception :

— **Double vulnérabilité :** $2{,}0 \times 2{,}0 = \mathbf{4{,}0}$ (dégâts quadruplés).
— **Compensation neutre :** $2{,}0 \times 0{,}5 = \mathbf{1{,}0}$ (dégâts neutres).
— **Double résistance :** $0{,}5 \times 0{,}5 = \mathbf{0{,}25}$ (dégâts divisés par 4).
— **Immunité stricte :** Si $M(T_{\text{att}}, T_{\text{déf},1}) = 0{,}0$, alors $M_{\text{total}} = \mathbf{0{,}0}$ (dégâts nuls, l'attaque échoue).

Déroulement Déterministe et Stochastique d'un Tour

> **i** Règle Fondamentale de l'Attaque dans le Socle Commun
>
> Dans le socle commun obligatoire, **les combattants ne choisissent pas leur attaque**. Chaque combattant porte une attaque standard unique dont le type élémentaire est **strictement son type principal** ($T_{\text{att}} = T_1$) et dont la précision est garantie à 100%. Seules les extensions 4 (Choix tactique) et 6 (Gestion des PP) modifient cette règle.

Soient deux combattants actifs C_1 et C_2. La résolution d'un tour de jeu suit la séquence chronologique ordonnée :

1. Phase d'Initiative

— Le combattant possédant la vitesse V la plus élevée frappe en premier.
— En cas d'égalité stricte ($V_1 = V_2$), le premier attaquant est désigné par un tirage de Bernoulli symétrique :

$$B \sim \mathcal{B}(0.5)$$

2. Phase de Frappe de l'Attaquant

L'attaquant actif assène son attaque de base (puissance nominale fixée à $P = 50$).

La formule mathématique fermée des dégâts réels infligés est :

$$\text{Dégâts} = \max\left(1, \left\lfloor \left(\frac{22 \times P \times \frac{A_{\text{att}}}{D_{\text{déf}}}}{50} + 2 \right) \times M_{\text{total}} \times U \times K \right\rfloor \right)$$

avec :

— $\lfloor x \rfloor$: opérateur partie entière inférieure.

— M_{total} : multiplicateur d'affinité élémentaire ($M(T_1, T_{\text{déf}})$ pour mono-type, produit croisé pour double type). Si $M_{\text{total}} = 0{,}0$, alors Dégâts $= 0$.
— U : facteur d'aléa d'impact, $U \sim \mathcal{U}([0.85, 1.00])$.
— K : multiplicateur de coup critique gouverné par une variable de Bernoulli $X_{\text{crit}} \sim \mathcal{B}(0.10)$:

$$K = \begin{cases} 1.5 & \text{si } X_{\text{crit}} = 1 \text{ (probabilité 10\%)} \\ 1.0 & \text{sinon} \end{cases}$$

3. Phase de Résolution et Riposte

$$\text{PV}_{\text{déf}} \leftarrow \max(0, \text{PV}_{\text{déf}} - \text{Dégâts})$$

— **Cas K.-O. :** Si $\text{PV}_{\text{déf}} = 0$, le défenseur s'effondre. Le match prend fin **instantanément**. Le défenseur ne riposte pas.
— **Cas Survie :** Si $\text{PV}_{\text{déf}} > 0$, les rôles s'inversent : le défenseur devient attaquant et exécute à son tour les étapes 2 et 3.

Architecture Logicielle et Standards de Développement

Arborescence Normée du Dépôt Git

L'arborescence du projet doit respecter rigoureusement la structure suivante :

```
pyarena/
   pyproject.toml              # Configuration du package, dépendances, pytest et linters
   README.md                   # Documentation technique et guide d'installation
   CONTRIBUTIONS.md            # Tableau contractuel des contributions individuelles
   data/
      types.csv                # Matrice des coefficients élémentaires (5 types)
      arenes_occitanie.geojson# Données spatiales régionales
      cache/                   # Fichiers JSON de l'API (ignorés par Git via .gitignore)
   src/
      pyarena/
          __init__.py          # Exports publics du package
          core/
              __init__.py
              exceptions.py    # Hiérarchie des exceptions maison (PyArenaException, etc.)
              models.py        # Classes Fighter, BaseFighter, BattleResult
              engine.py        # Moteur déterministe d'exécution des duels tour par tour
          io/
              __init__.py
              fetcher.py       # Client HTTP PokéAPI avec mise en cache disque
              loader.py        # Parsers de fichiers CSV et GeoJSON
          simulation/
              __init__.py
```

```
            monte_carlo.py  # Simulateur Monte-Carlo distribué (multiprocessing/joblib)
        extensions/         # Code source exclusif de l'extension attribuée
            __init__.py
tests/
    __init__.py
    test_models.py          # Tests des entités, du typage et des propriétés
    test_engine.py          # Tests unitaires du calcul de dégâts et du cycle de tour
    test_simulation.py      # Tests de reproductibilité (seed) et de parallélisation
    test_extension.py       # Tests unitaires exhaustifs de l'extension choisie
app/
    main.py                 # Point d'entrée de l'application Streamlit
    pages/                  # Pages multipages de l'interface
    utils.py                # Fonctions d'affichage graphique (Plotly, Folium)
```

Règles d'Or de la Programmation Orientée Objet (POO)

1. **Rôle précis des classes du socle commun (`core/models.py`) :**
   - `BaseFighter` (classe abstraite héritant de `abc.ABC`) : déclare l'interface contractuelle avec `@abstractmethod` : `take_damage(amount)`, `is_alive()`, `reset_stats()`.
   - `Fighter` (classe concrète héritant de `BaseFighter`) : implémente un combattant individuel avec ses attributs encapsulés et validés par `@property`.
   - `BattleResult` (dataclass ou classe immuable) : structure de retour standardisée d'un duel consignant le vainqueur, le perdant, le nombre de tours et l'état des PV restants.
2. **Encapsulation stricte :**
   - L'attribut de santé `_hp` est strictement privé ou protégé.
   - Le setter `@hp.setter` interdit formellement toute valeur incohérente : une valeur négative est ramenée à 0, et un dépassement de $\mathrm{PV}_{\max}$ est borné.
3. **Méthodes Spéciales (*Dunder methods*) :**
   - Implémenter obligatoirement `__repr__` (signature technique non ambiguë), `__str__` (affichage lisible) et `__eq__` (égalité structurelle basée sur `id`).
4. **Gestion Robuste des Exceptions (`core/exceptions.py`) :**
   - Classe racine : `PyArenaException(Exception)`.
   - Classes dérivées obligatoires :
     - `FighterFaintedError` : levée lorsqu'une entité à PV = 0 tente d'agir.
     - `InvalidStatError` : levée si une statistique extraite est négative ou nulle.
     - `DataFetchError` : levée si l'API est injoignable et qu'aucun cache local n'existe sur le disque.

Packaging et Intégration Continue (GitHub Actions)

- Le dépôt doit être installable en mode éditable à la racine :

```
pip install -e .
```

— Le workflow `.github/workflows/ci.yml` s'exécute à chaque commit sur `main` et à chaque Pull Request :

1. Contrôle du style et du formatage avec `ruff check`.
2. Exécution de la suite de tests avec `pytest`.
3. Mesure de couverture avec `pytest-cov` :
   — **Équipes de 3 :** Échec immédiat si couverture globale $< 70\%$.
   — **Équipes de 4 :** Échec immédiat si couverture globale $< 85\%$.

Pipeline de Données, Cache Local et Simulation

Ingestion PokéAPI et Cache Hors-Ligne

— Le module `io/fetcher.py` implémente `DataFetcher` requêtant l'API ouverte `https://pokeapi.co/api/v2/pok`
— **Politique de cache disque :** Tout fichier JSON reçu est sérialisé sous `data/cache/{id}.json`.
— Toute exécution ultérieure lit prioritairement le fichier local sans solliciter le réseau.
— Le dossier `data/cache/` doit figurer dans le fichier `.gitignore` : **interdiction absolue de pousser les fichiers JSON sur GitHub**.

Moteur de Simulation Monte-Carlo Parallélisé

— La fonction fondamentale `simulate_battle(f1, f2, seed=None)` permet l'injection d'une graine aléatoire pour reproduire un affrontement à l'identique.
— Le module `simulation/monte_carlo.py` expose la fonction :

```
def run_monte_carlo(
    f1: BaseFighter,
    f2: BaseFighter,
    n_simulations: int = 5000,
    n_jobs: int = -1
) -> dict:
```

— **Distribution multi-cœurs :** Les $n_{\text{simulations}}$ duels doivent être distribués via `multiprocessing.Pool` ou `joblib.Parallel`.
— **Indicateurs retournés dans le dictionnaire de synthèse :**

1. Winrate ponctuel de C_1 :
$$\hat{p} = \frac{N_{\text{victoires de } C_1}}{n_{\text{simulations}}}$$
2. Intervalle de confiance asymptotique à 95 % :
$$\text{IC}_{0.95}(\hat{p}) = \left[\hat{p} - 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n_{\text{simulations}}}},\ \hat{p} + 1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{n_{\text{simulations}}}}\right]$$

3. Statistique descriptive de la durée du match (nombre de tours jusqu'au K.-O.) : moyenne, médiane, écart-type.

Spécifications du Tableau de Bord Streamlit

L'application Streamlit (`app/main.py`) constitue le livrable visuel décisionnel.

Module 1 : Simulateur de Duel et Télémétrie en Direct (Commun)

— Listes déroulantes pour sélectionner les combattants A et B.
— Affichage immédiat des sprites officiels (champ `sprites.front_default` de l'API) et des statistiques normalisées.
— Curseur dynamique réglant le nombre de duels ($100 \leq N \leq 10\,000$).
— Restitution post-calcul : jauges de winrate, intervalle de confiance à 95 %, histogramme Plotly de la durée des combats et journal textuel d'un match témoin généré avec graine fixée.

Module 2 : Catalogue et Analyse Exploratoire des Données (Commun)

— Tableau interactif filtrable par type et caractéristiques.
— Nuage de points Plotly : *Attaque* vs *Défense*, taille proportionnelle aux PV, couleur selon le type principal.
— Boîtes à moustaches (*boxplots*) comparant la distribution de la vitesse entre les types.

Module 3 : Cartographie Géospatiale des Arènes d'Occitanie (Commun)

— Exploitation de `data/arenes_occitanie.geojson` recensant 30 arènes régionales (Montpellier, Béziers, Nîmes, Toulouse…).
— Carte dynamique via `streamlit-folium` avec marqueurs colorés selon le type de l'arène.
— Clic sur un marqueur : pop-up détaillant le champion local, son sprite et ses statistiques.

Module 4 : Analyseur de Synergie d'Équipe (Obligatoire Groupes de 4 / Bonus Groupes de 3)

Outil décisionnel analysant un trio de combattants $E = \{C_1, C_2, C_3\}$:

1. **Bilan des Vulnérabilités Défensives :**
   Pour chacun des 18 types $t \in \mathcal{T}$, calcule combien de membres subissent des dégâts critiques ($M_{\text{total}} \geq 2{,}0$). Alerte visuelle rouge si ≥ 2 membres partagent la même faiblesse.
2. **Couverture Offensive Globale :**
   Calcule le pourcentage des 18 types adverses frappés avec un avantage ($M > 1{,}0$) par au moins un membre :
   $$\text{Taux de Couverture} = \frac{\sum_{t \in \mathcal{T}} \mathbb{1}_{\{\exists C \in E \,:\, M(T_1(C), t) > 1{,}0\}}}{18} \times 100$$
   Visualisation sous forme de diagramme radar Plotly.

Spécifications Exhaustives des 10 Extensions Thématiques

Chaque équipe de projet implémente **une seule extension exclusive** parmi les dix présentées ci-après.

Cadre Technique Commun et Équité

Toutes les extensions ont été calibrées pour mobiliser une charge de travail strictement homogène :

- **Développement métier :** Lignes de code Python réparties dans un module dédié sous `src/pyarena/extensions/`.
- **Assurance qualité :** Une suite de tests unitaires stricts dans `tests/test_extension.py` couvrant les propriétés nominales, les cas aux limites et les propriétés algébriques ou statistiques.
- **Restitution décisionnelle :** Un volet dédié dans Streamlit articulant une **Vue Macro** (agrégation statistique sur $N = 5\,000$ duels) ou/et une **Vue Micro** (télémétrie d'un combat singulier avec graine fixée).

Extension 1 : Objets Passifs et Équipements Stratégiques

- **Fichier source :** `src/pyarena/extensions/items.py`
- **Problématique décisionnelle :** Modéliser l'impact d'équipements passifs sur le rééquilibrage de duels asymétriques et quantifier leur capacité à neutraliser un désavantage de statistiques ou d'affinité élémentaire.
- **Spécifications logicielles et POO :**
  - Concevoir une classe abstraite `BaseItem(ABC)` dotée par exemple de trois méthodes polymorphiques :

```python
class BaseItem(ABC):
    @abstractmethod
    def apply_passive(self, fighter: BaseFighter) -> None:
        """Ajuste les attributs initiaux du combattant en début de duel."""
        pass

    @abstractmethod
    def on_damage_received(
        self, incoming_damage: int, current_hp: int
    ) -> int:
        """Intercepte et modifie les dégâts subis avant leur déduction."""
        pass

    @abstractmethod
    def on_turn_end(self, fighter: BaseFighter) -> None:
        """Exécute les effets séquentiels en clôture de tour (soin, contrecoup)."""
        pass
```

  - Implémenter obligatoirement au moins quatre classes concrètes d'objets :

1. `FocusSash` (*Ceinture Force*) : Si PV $=$ PV_{max} et que Dégâts $\geq$ PV, les dégâts entrants sont plafonnés à $\text{PV}_{\text{max}}-1$, laissant le porteur à PV $= 1$. Possède un attribut d'état `is_consumed: bool = False` qui bascule à `True` dès l'activation. L'objet devient inopérant pour le reste du match.
2. `SitrusBerry` (*Baie Sitrus*) : Possède l'attribut `is_consumed: bool = False`. Dès que PV $\leq \lfloor 0{,}50 \times \text{PV}_{\text{max}} \rfloor$, restitue immédiatement $\lfloor 0{,}25 \times \text{PV}_{\text{max}} \rfloor$ PV. Le soin est plafonné à PV_{max}. L'objet bascule à `is_consumed = True` et ne peut plus se déclencher.
3. `LifeOrb` (*Orbe Vie*) : Applique un multiplicateur fixe de $\times 1{,}30$ sur l'ensemble des dégâts physiques infligés par le porteur. En contrepartie, après chaque frappe assénée, le porteur subit un contrecoup de $\lfloor 0{,}10 \times \text{PV}_{\text{max}} \rfloor$ PV. Ce recul peut conduire au K.-O. du lanceur.
4. `Leftovers` (*Restes*) : Dans la méthode `on_turn_end`, régénère $\lfloor \frac{\text{PV}_{\text{max}}}{16} \rfloor$ PV à la clôture de chaque tour, plafonné à PV_{max}.

— **Cas aux limites et règles de gestion :**

— Si un combattant équipé de la *Ceinture Force* subit une attaque fatale alors qu'il a déjà perdu au moins 1 PV (PV $< \text{PV}_{\text{max}}$), la protection échoue et l'entité tombe K.-O.
— Si l'activation de l'*Orbe Vie* met le défenseur et l'attaquant à 0 PV sur le même tour, l'attaquant est déclaré vainqueur (les dégâts sont appliqués avant le contrecoup).

— **Tests unitaires obligatoires (`tests/test_extension.py`) :**

— `test_focus_sash_full_hp` : Valider la survie à 1 PV lors d'une attaque mortelle à PV pleins[cite : 1].
— `test_focus_sash_damaged` : Valider l'inactivité de la Ceinture Force si $\text{PV}_{\text{initial}} = \text{PV}_{\text{max}} - 1$.
— `test_sitrus_berry_threshold_and_consumption` : Vérifier que le soin ne s'active pas à 51% de PV, se déclenche à 50%, et ne se répète jamais une seconde fois.
— `test_life_orb_recoil_ko` : Vérifier la majoration arithmétique de 30% des dégâts et la possibilité de mourir du contrecoup.
— `test_leftovers_cap` : Vérifier que les Restes ne permettent aucun dépassement de PV_{max}.

— **Composant Streamlit :**

— *Vue Macro :* Heatmap Plotly 5×5 affichant la matrice du delta de winrate $\Delta\hat{p} = \hat{p}_{\text{équipé}} - \hat{p}_{\text{sans}}$ pour chaque combinaison d'objets sur un duel de référence.
— *Vue Micro :* Télémétrie d'un match témoin avec mise en avant graphique des activations d'objets (pastilles vertes de régénération, bouclier doré pour la Ceinture Force).

Extension 2 : Combats d'Équipes 3 contre 3 avec Relais (Tag-Team)

— **Fichier source :** `src/pyarena/extensions/teams.py`
— **Problématique décisionnelle :** Modéliser une confrontation séquentielle par équipes et déterminer l'ordonnancement d'alignement optimal (*matchup sequencing*) face à une composition adverse donnée.
— **Spécifications logicielles et POO :**

— Concevoir une classe `Team` encapsulant une file (FIFO) de trois instances concrètes de `BaseFighter` :

```
class Team:
    def __init__(self, fighters: list[BaseFighter]):
        if len(fighters) != 3:
            raise ValueError("Une équipe doit comporter exactement 3 combattants.")
```

```
        self._queue: list[BaseFighter] = list(fighters)

    def get_active(self) -> BaseFighter:
        return self._queue[0]

    def pop_defeated(self) -> BaseFighter:
        return self._queue.pop(0)

    def is_defeated(self) -> bool:
        return len(self._queue) == 0
```

— Concevoir une classe `TeamBattle` orchestrant la rencontre :

  — Les combattants en tête de file débutent le combat.
  — Dès qu'un combattant actif est K.-O., il est retiré de la file. Le combattant suivant entre immédiatement dans l'arène avec ses PV pleins face à l'adversaire survivant.
  — Le combattant survivant conserve scrupuleusement ses points de vie résiduels (aucune réinitialisation entre les duels).
  — Condition de victoire : l'affrontement cesse dès que l'une des deux équipes a ses 3 membres K.-O.

— Fonction d'optimisation d'ordre d'alignement :

```
def optimize_team_order(
    my_team: list[BaseFighter],
    opponent_team: list[BaseFighter],
    n_simulations: int = 1000,
) -> dict[tuple, float]:
    """Évalue les 3! = 6 permutations d'alignement de my_team face à l'adversaire."""
```

— **Cas aux limites et règles de gestion :**

  — L'initiative (qui frappe en premier) est recalculée à chaque entrée d'un nouveau combattant selon la comparaison stricte des vitesses V.
  — Les statistiques modifiées en cours de duel ne se transmettent pas au combattant suivant de la file.

— **Tests unitaires obligatoires (`tests/test_extension.py`) :**

  — `test_relay_transition` : Vérifier que le combattant n°2 prend le relais au tour exact suivant le K.-O. du combattant n°1.
  — `test_hp_persistence_between_rounds` : Contrôler que le vainqueur d'un duel intermédiaire conserve rigoureusement ses PV entamés face au nouvel arrivant.
  — `test_battle_termination_at_three_ko` : Valider la clôture stricte du match après trois K.-O. dans la même équipe.
  — `test_optimization_permutations` : Valider que `optimize_team_order` évalue bien les 6 permutations et renvoie des probabilités comprises dans $[0, 1]$.

— **Composant Streamlit :**

  — *Vue Macro :* Diagramme en barres Plotly classant les 6 permutations d'alignement possibles par winrate décroissant, mettant en valeur l'alignement optimal σ^* et le différentiel de victoires par rapport à la pire permutation.
  — *Vue Micro :* Suivi graphique en direct des 6 jauges de santé qui diminuent de façon synchrone au fil des relais d'un match témoin.

Extension 3 : Matrice Complète de Méta-Jeu et Détection d'Intransitivité

- **Fichier source :** `src/pyarena/extensions/round_robin.py`
- **Problématique décisionnelle :** Identifier l'existence de cycles d'intransitivité généralisés (équilibres de type « Pierre-Feuille-Ciseaux ») au sein d'un panel de combattants et formaliser la structure topologique du méta-jeu.
- **Spécifications logicielles et Algorithmes :**
  - Isoler une liste fixe de 10 combattants représentatifs aux profils de types et de statistiques diversifiés.
  - Orchestrer un tournoi toutes rondes (*Round-Robin*) : simuler l'ensemble des $\frac{10\times 9}{2} = 45$ duels non redondants via le moteur Monte-Carlo ($N = 1\,500$ simulations par cellule).
  - Construire la matrice d'adjacence stochastique $M \in \mathbb{R}^{10\times 10}$ où $M_{i,j} = \widehat{p}(C_i \text{ bat } C_j)$.
  - **Détection formelle des cycles d'intransitivité :**
    - Soit la relation de dominance empirique :

$$C_i \succ C_j \iff M_{i,j} > 0{,}50$$

    - Parcourir l'ensemble des $\binom{10}{3} = 120$ combinaisons non ordonnées de triplets $\{C_i, C_j, C_k\}$ pour extraire la totalité des 3-cycles orientés vérifiant :

$$C_i \succ C_j \quad \text{et} \quad C_j \succ C_k \quad \text{et} \quad C_k \succ C_i$$

    - Renvoyer les cycles détectés sous forme d'une liste de tuples de noms `[(str, str, str)]`.
- **Cas aux limites et règles de gestion :**
  - Si $M_{i,j} = 0{,}50$ (égalité stochastique stricte), la relation de dominance n'est pas établie et la paire ne peut participer à aucun cycle.
- **Tests unitaires obligatoires (`tests/test_extension.py`) :**
  - `test_matrix_symmetry_properties` : Vérifier que $M_{i,i} = 0{,}50$ et que pour tout $i \neq j$, $|(M_{i,j} + M_{j,i}) - 1{,}0| < 10^{-4}$.
  - `test_cycle_detection_synthetic` : Injecter une matrice 3×3 d'intransitivité pure (A bat B à 70 %, B bat C à 70 %, C bat A à 70 %) et valider que l'algorithme extrait exactement le cycle (A, B, C).
  - `test_no_cycle_in_linear_hierarchy` : Valider que sur une matrice représentant une hiérarchie stricte ($A \succ B \succ C$ avec $A \succ C$), aucun cycle n'est renvoyé.
- **Composant Streamlit :**
  - *Heatmap Plotly :* Représentation 10×10 munie d'une échelle de couleurs divergente centrée sur 0,50.
  - *Graphe Orienté Circulaire :* Affichage des 10 nœuds disposés en cercle. Pour préserver la lisibilité, le graphe n'affiche par défaut que les arêtes de forte dominance ($\widehat{p} \geq 0{,}60$).
  - *Sélecteur de Cycles :* Composant `st.selectbox` énumérant les cycles d'intransitivité détectés. Sélectionner un cycle efface les autres arêtes du graphe et illumine en rouge vif les 3 flèches du cycle avec 3 colonnes `st.metric` associées.

Extension 4 : Choix Stratégique d'Attaques (Précision vs Puissance)

- **Fichier source :** `src/pyarena/extensions/moves.py`

— **Problématique décisionnelle :** Modéliser la prise de décision séquentielle en univers incertain en arbitrant entre rendement espéré maximal et minimisation du risque de défaillance immédiate.
— **Spécifications logicielles et Algorithmes :**

— Doter chaque combattant de deux capacités alignées sur son type principal T_1 :

1. *Frappe Sûre* : Puissance $P = 40$, Précision $\alpha = 1{,}00$ (succès garanti).
2. *Frappe Risquée* : Puissance $P = 85$, Précision $\alpha = 0{,}65$ (35 % de chances d'échec total, infligeant 0 dégât).

— Définir trois agents décisionnels sélectionnables :

— `ConservativeAgent` : Sélectionne systématiquement la Frappe Sûre.
— `AggressiveAgent` : Sélectionne systématiquement la Frappe Risquée.
— `OptimalExpectationAgent` : Calcule à chaque tour les dégâts théoriques moyens des deux options en intégrant les statistiques du défenseur et les multiplicateurs M_{total} :

$$\mathbb{E}[\text{Dégâts}_{\text{sûre}}] = 1{,}00 \times \text{Dégâts}(P = 40, M_{\text{total}})$$
$$\mathbb{E}[\text{Dégâts}_{\text{risquée}}] = 0{,}65 \times \text{Dégâts}(P = 85, M_{\text{total}})$$

Règle du Coup de Grâce : Si $\text{Dégâts}(P = 40, M_{\text{total}}) \geq \text{PV}_{\text{cible}}$, l'agent choisit **obligatoirement** la Frappe Sûre (garantie de K.-O. à 100 % au lieu de s'exposer à 35 % d'échec). Sinon, il choisit l'option d'espérance maximale.

— **Cas aux limites et règles de gestion :**

— En cas d'échec d'une Frappe Risquée, aucun dégât n'est infligé, le coup critique ne peut pas être calculé et le tour de l'attaquant est consommé.
— **Tests unitaires obligatoires (`tests/test_extension.py`) :**

— `test_risky_move_failure_rate` : Vérifier sur 10 000 itérations que le taux d'échec empirique converge vers $0{,}35 \pm 0{,}015$.
— `test_risky_move_zero_damage` : Valider qu'une attaque échouée maintient les PV de la cible strictement inchangés.
— `test_coup_de_grace_override` : Vérifier formellement que l'agent par espérance bascule sur la Frappe Sûre dès que la cible est à portée de frappe certaine, même si $\mathbb{E}[\text{Dégâts}_{\text{risquée}}] > \mathbb{E}[\text{Dégâts}_{\text{sûre}}]$.
— **Composant Streamlit :**

— *Vue Macro :* Histogrammes superposés Plotly de la distribution de la durée des matchs (nombre de tours jusqu'au K.-O.) comparant les confrontations Conservateur vs Conservateur, Agressif vs Agressif et Espérance vs Agressif.
— *Vue Micro :* Télémétrie pas-à-pas consignant la décision prise par l'agent décisionnel et la justification mathématique de son arbitrage tour par tour.

Extension 5 : Système Stochastique d'Altérations d'État

— **Problématique décisionnelle :** Modéliser des "chaînes de Markov" avec états absorbants pour quantifier comment des pénalités d'état aléatoires déstabilisent l'avantage initial de vitesse et d'attaque.
— **Spécifications logicielles :**

— Définir l'énumération `Status(Enum)` dans `src/pyarena/extensions/statuses.py` : `NONE`, `BURN`, `PARALYSIS`, `POISON`.
— Chaque attaque réussie a $p = 0{,}15$ de chances d'induire une altération sur une cible saine, dictée par le type principal T_1 de l'attaquant :

  — **Feu → Brûlure (`BURN`) :** L'attaque du porteur est divisée par deux de façon permanente ($A \leftarrow \lfloor A/2 \rfloor$). En clôture de chaque tour, il perd $\lfloor \mathrm{PV}_{\max}/16 \rfloor$ PV.
  — **Électrik → Paralysie (`PARALYSIS`) :** La vitesse du porteur est divisée par deux de façon permanente ($V \leftarrow \lfloor V/2 \rfloor$). Au début de chaque tour, le porteur subit un tirage de Bernoulli $B \sim \mathcal{B}(0{,}25)$: si $B = 1$, le combattant est tétanisé et son tour est annulé.
  — **Plante ou Poison → Poison (`POISON`) :** En fin de chaque tour, le porteur perd passivement $\lfloor \mathrm{PV}_{\max}/8 \rfloor$ PV.
— **Règles d'exclusivité et de rémanence :** Un combattant ne peut cumuler plusieurs statuts simultanément. Une altération persiste jusqu'au terme du duel (aucune guérison spontanée). Les combattants d'autres types n'infligent aucun statut mais peuvent les subir.

— **Tests unitaires (`tests/test_extension.py`) :**

  — Vérifier la division exacte par deux de A et V dès l'application respective de la Brûlure et de la Paralysie.
  — Vérifier l'immunité aux autres statuts d'un combattant déjà altéré.
  — Valider que les dégâts de fin de tour décrémentent fidèlement les points de vie et peuvent provoquer un K.-O.

— **Composant Streamlit :**

  — *Vue Macro :* Histogramme Plotly superposant la distribution de la durée des duels (nombre de tours) avec et sans statuts activés, complété par trois cartes `st.metric` affichant la probabilité de victoire globale $\hat{p}(\text{Victoire})$ et la probabilité conditionnelle $\hat{p}(\text{Victoire} \mid \text{Statut appliqué dans les 3 premiers tours})$.
  — *Vue Micro :* Badges de statut colorés au-dessus des sprites des combattants et affichage explicite dans le journal textuel des tours annulés pour cause de paralysie.

Extension 6 : Gestion des Points de Puissance (PP) et Action Lutte

— **Problématique décisionnelle :** Modéliser la dégradation des performances offensives induite par la raréfaction des ressources et analyser la dynamique de survie en combat d'usure.
— **Spécifications logicielles :**

  — Concevoir une classe `Move` dans `src/pyarena/extensions/skills.py` encapsulant nom, puissance et capital de Points de Puissance (`pp_max`, `pp_current`).
  — Doter chaque combattant de deux capacités alignées sur son type principal T_1 :

    1. *Capacité Majeure* : Puissance $P = 75$, $\mathrm{PP}_{\max} = 5$ utilisations.
    2. *Capacité Standard* : Puissance $P = 45$, $\mathrm{PP}_{\max} = 15$ utilisations.

  — Ordonnancement automatique : Le combattant consomme sa Capacité Majeure tant que PP ≥ 1, puis bascule sur sa Capacité Standard jusqu'à épuisement.
  — **Action par défaut (*Lutte*) :** Dès que le total cumulé des PP tombe à 0, le combattant exécute obligatoirement l'action *Lutte* :

    — Puissance nominale $P = 50$.

  - Type strictement **neutre** : applique systématiquement $M_{\text{total}} = 1{,}0$ face à n'importe quel adversaire, ignorant faiblesses et immunités.
  - Contrecoup immédiat : l'attaquant subit une perte de $\lfloor 0{,}25 \times \text{Dégâts réels infligés à la cible} \rfloor$ PV.
- *Règle du double K.-O. :* Si Lutte met le défenseur à 0 PV mais que le contrecoup met aussi l'attaquant à 0 PV, l'attaquant est déclaré vainqueur (la cible est tombée en premier sous l'impact).

— **Tests unitaires (`tests/test_extension.py`) :**

- Vérifier la décrémentation des PP et la bascule exacte de la Capacité Majeure vers la Standard au tour 6.
- Valider le basculement forcé vers Lutte dès le 21 tour d'attaque du combattant.
- Vérifier la neutralité élémentaire stricte de Lutte face à des types présentant des immunités ou des doubles résistances.
- Valider que le recul retiré au lanceur correspond rigoureusement à 25% des dégâts infligés.

— **Composant Streamlit :**

- *Vue Macro :* Diagramme en barres Plotly comparant le taux d'activation du mode Lutte (en %) sur un panel de 5 confrontations types (duels rapides offensifs vs duels d'usure défensifs), et graphique circulaire montrant la répartition des dégâts cumulés (Capacité Majeure vs Capacité Standard vs Recul subi).
- *Vue Micro :* Deux jauges de progression interactives (`st.progress`) retraçant la consommation des stocks de PP pour un match témoin sélectionné.

Extension 7 : Accélération Vectorisée et Profilage Numérique (NumPy / Numba)

— **Fichier source :** `src/pyarena/extensions/fast_sim.py`

— **Problématique d'ingénierie logicielle :** Mettre en œuvre le patron d'architecture *Bridge/Kernel* pour dépasser les verrous de performance de la POO et simuler des millions de combats à l'échelle industrielle.

— **Spécifications logicielles et Compilation JIT :**

- Profiler l'exécution de 10 000 combats objets standard à l'aide de `cProfile` pour quantifier le coût d'accès aux attributs et de la POO Python.
- Concevoir un moteur bas niveau sans objet compilé à la volée avec Numba :
  - Projeter chaque combattant dans un tableau NumPy 1D contigu (`np.float64`) :

$$\text{state} = [\text{PV}_{\max}, \text{PV}, A, D, V, \text{type_id}]$$

  - Encoder la table des multiplicateurs élémentaires sous forme d'une matrice NumPy 2D contiguë `type_matrix` de dimensions 5×5 (ou 18×18).
  - Réécrire la boucle complète de combat au sein d'une fonction numérique pure :

```python
@njit(fastmath=True)
def simulate_battle_numba(
    s1: np.ndarray, s2: np.ndarray, type_matrix: np.ndarray, seed: int
) -> int:
    """Retourne 1 si le combattant 1 gagne, 0 sinon."""
```

— Écrire la fonction de simulation par lots :

```python
@njit(fastmath=True)
def run_monte_carlo_numba(
    s1: np.ndarray,
    s2: np.ndarray,
    type_matrix: np.ndarray,
    n_sims: int,
    base_seed: int,
) -> float:
    """Exécute n_sims duels en code machine compilé et renvoie le winrate."""
```

— Mesurer le facteur d'accélération pour des tailles d'échantillons $N \in [10^2, 10^6]$:

$$\text{Speedup}(N) = \frac{T_{\text{objet}}(N)}{T_{\text{numba}}(N)}$$

— **Cas aux limites et règles de gestion :**

— La fonction Numba ne doit contenir aucun appel à des structures Python dynamiques (pas de dictionnaires, pas d'objets, pas de chaînes de caractères), sous peine d'échouer à la compilation `nopython mode`.

— **Tests unitaires obligatoires (`tests/test_extension.py`) :**

— `test_numba_compilation_success` : Vérifier que la fonction compilée s'exécute sans lever de `TypingError`.

— `test_statistical_equivalence` : Valider sur 5 000 itérations avec graine fixée que l'estimation du winrate produit par Numba est identique à celle du moteur objet à $\pm 1{,}5\%$ près.

— `test_deterministic_behavior` : Vérifier que deux exécutions Numba avec la même graine renvoient le même nombre de victoires.

— **Composant Streamlit :**

— *Vue Performance :* Graphique interactif Plotly en échelle semi-logarithmique affichant les courbes de temps de calcul absolu et le facteur d'accélération résultant en fonction de N.

— *Vue Profilage :* Tableau de synthèse des métriques `cProfile` isolant le coût des allocations d'objets.

Extension 8 : Modélisation Paramétrique de Niveaux et Détermination de Seuil Critique

— **Fichier source :** `src/pyarena/extensions/levels.py`

— **Problématique décisionnelle :** Modéliser la dynamique paramétrique des attributs et concevoir un algorithme de recherche discrète convergeant vers le point de bascule stochastique face à des profils types.

— **Spécifications logicielles et Modélisation :**

— Implémenter le recalcul des caractéristiques pour tout niveau $L \in [\![1, 100]\!]$ selon les équations officielles de Game Freak :

$$\text{PV}(L) = \left\lfloor \frac{2 \times \text{PV}_{\text{base}} \times L}{100} \right\rfloor + L + 10$$

$$\text{Stat}(L) = \left\lfloor \frac{2 \times \text{Stat}_{\text{base}} \times L}{100} \right\rfloor + 5 \quad (\text{pour } A, D, V)$$

— **Optimisation de la recherche de racine stochastique :** Soit la fonction de winrate $L \mapsto \hat{p}(L)$, monotone croissante d'allure sigmoïde, mesurant la performance d'un combattant $C_1(L)$ face à un adversaire fixé au niveau 50 standard.
Concevoir un algorithme de recherche dichotomique discrète convergeant vers le **Niveau Critique** L^* :

$$L^* = \min\{L \in [\![1, 100]\!] \mid \hat{p}(L) \geq 0{,}50\}$$

L'algorithme doit opérer en un nombre d'étapes limité au lieu de balayer séquentiellement les 100 niveaux.

— **Benchmark multi-profils :** L'évaluation doit confronter $C_1(L)$ à quatre archétypes d'adversaires de niveau 50 :

1. *Archétype Équilibré* (statistiques moyennes, type neutre).
2. *Archétype Tank Défensif* (PV et D très élevés).
3. *Archétype Attaquant Rapide* (A et V dominants).
4. *Archétype Défavorable* (adversaire bénéficiant d'un multiplicateur élémentaire $\times 2{,}0$).

— **Cas aux limites et règles de gestion :**

— Si même au niveau $L = 100$, le winrate reste strictement inférieur à 0,50, l'algorithme renvoie $L^* = \text{None}$ (dominance impossible).

— **Tests unitaires obligatoires (`tests/test_extension.py`) :**

— `test_stat_scaling_exact_values` : Valider la conformité des statistiques calculées aux niveaux $L = 1$, $L = 50$ et $L = 100$ par rapport à des valeurs de contrôle calculées à la main.
— `test_strict_monotonicity` : Vérifier que $\text{PV}(L+1) \geq \text{PV}(L)$ et $\text{Stat}(L+1) \geq \text{Stat}(L)$ pour tout $L \in [\![1, 99]\!]$.
— `test_binary_search_convergence` : Vérifier que la recherche dichotomique renvoie bien un seuil vérifiant $\hat{p}(L^*) \geq 0{,}50$ et $\hat{p}(L^* - 1) < 0{,}50$ sur un duel témoin.

— **Composant Streamlit :**

— *Vue Synthèse :* Superposition graphique Plotly des 4 courbes sigmoïdes $L \mapsto \hat{p}(L)$ évaluées par pas de 5 ($L \in \{5, 10, \ldots, 100\}$) pour les 4 archétypes.
— *Vue Métrique :* Ligne verticale marquant le seuil L^* et son intervalle de confiance à 95 %.

Extension 9 : La Route des Champions (Occitanie — TSP sous Contraintes de Niveau)

— **Problématique d'optimisation :** Modéliser et résoudre le problème du voyageur de commerce (TSP) sous contraintes d'ordre partiel sur les 30 arènes régionales d'Occitanie, et quantifier le surcoût kilométrique imposé par une progression ordonnée par niveau de difficulté.

— **Spécifications logicielles :**

— Exploiter le fichier spatial `data/arenes_occitanie.geojson` contenant les coordonnées géographiques $(\text{lat}_i, \text{lon}_i)$, le nom et le niveau de difficulté (échelonné de 15 à 60) des 30 arènes régionales.
— Implémenter le calcul de la distance orthodromique de Haversine $d(u, v)$ en kilomètres entre deux coordonnées :

$$\Delta\varphi = \varphi_2 - \varphi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1$$

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$d = 2R \arcsin\left(\sqrt{a}\right) \quad \text{avec } R = 6\,371 \text{ km}$$

— Implémenter et comparer trois algorithmes de circuit fermé (départ et retour fixés à Montpellier) :

1. *Algo 1 — Heuristique Gloutonne Libre (Plus Proche Voisin) :* Construit un cycle initial T_{glouton} en visitant itérativement le sommet non visité le plus proche géographiquement, sans considération de niveau.
2. *Algo 2 — Amélioration Locale 2-opt (Optimisation Libre) L'heuristique gloutonne du plus proche voisin produit une tournée initiale $T = (v_0, v_1, \dots, v_{n-1}, v_0)$ sous-optimale présentant de nombreux croisements d'arêtes sur le plan géographique. L'algorithme **2-opt** est une méthode de recherche locale qui élimine itérativement ces croisements en inversant des sous-parcours.

— **Principe géométrique d'inversion :** Soient deux arêtes non consécutives du cycle : (v_i, v_{i+1}) et (v_j, v_{j+1}) avec $0 \leq i < j - 1 < n$.
Si l'on supprime ces deux arêtes pour reconnecter v_i à v_j et v_{i+1} à v_{j+1}, le sous-parcours intermédiaire $(v_{i+1}, \dots, v_j)$ doit nécessairement être parcouru en sens inverse pour préserver un cycle fermé valide :

$$T_{\text{nouveau}} = (v_0, \dots, v_i, \underbrace{v_j, v_{j-1}, \dots, v_{i+1}}_{\text{sous-parcours inversé}}, v_{j+1}, \dots, v_{n-1}, v_0)$$

— **Critère de décroisement (Gain kilométrique) :** Pour toute paire d'arêtes (v_i, v_{i+1}) et (v_j, v_{j+1}), on calcule le différentiel de distance induit :

$$\Delta d = \Big(d(v_i, v_j) + d(v_{i+1}, v_{j+1})\Big) - \Big(d(v_i, v_{i+1}) + d(v_j, v_{j+1})\Big)$$

— Si $\Delta d < 0$: le remplacement raccourcit le trajet. L'inversion est validée immédiatement sur la liste en Python : `route[i+1 : j+1] = reversed(route[i+1 : j+1])`
— Si $\Delta d \geq 0$: les arêtes d'origine sont conservées.

— **Condition d'arrêt :** On répète le balayage de toutes les paires d'indices (i, j) tant qu'au moins une inversion améliorant la distance totale est détectée (`amelioration = True`). L'algorithme s'arrête lorsqu'un balayage complet n'apporte plus aucun gain ($\Delta d \geq 0$ pour toutes les paires), garantissant la convergence vers un minimum local sans aucun croisement d'arêtes sur la carte. La longueur totale du cycle obtenu constitue la référence minimale libre $d_{\textbf{libre}}$.

3. *Algo 3 — Tournée des Champions (Contrainte d'Ordre par Paliers) :* Partitionne les arènes en 4 paliers ordonnés : $P_1 = [15, 25]$, $P_2 = [26, 35]$, $P_3 = [36, 45]$, $P_4 = [46, 60]$.
Règle métier : Il est interdit de visiter une arène du palier P_{k+1} avant d'avoir vaincu l'ensemble des arènes du palier P_k. Le parcours optimise l'enchaînement intra-palier via 2-opt et relie séquentiellement $P_1 \to P_2 \to P_3 \to P_4 \to$ Montpellier, produisant une tournée de longueur $d_{\textbf{champions}}$.

— Calculer l'indicateur d'impact de la contrainte :

$$\text{Surcoût Kilométrique} = \frac{d_{\text{champions}} - d_{\text{libre}}}{d_{\text{libre}}} \times 100$$

— **Tests unitaires (`tests/test_extension.py`) :**

— Propriétés métriques de la distance de Haversine : positivité, symétrie stricte, nullité de la distance à soi-même et conformité sur une distance réelle connue (Montpellier–Nîmes $\approx 45 - 55$ km).

— Vérifier que l'algorithme 2-opt garantit une longueur strictement inférieure ou égale à l'heuristique gloutonne : $d_{\text{libre}} \leq d_{\text{glouton}}$.
— Contrôler que la séquence des villes de $T_{\text{champions}}$ respecte scrupuleusement l'ordre des 4 paliers de niveau.

— **Composant Streamlit :**

— *Carte Folium :* Tracé interactif des deux tournées superposées (`folium.PolyLine` avec code couleur distinct : bleu pour la tournée libre optimale, rouge pour la tournée des champions).
— *Tableau de bord métrique :* Comparaison des kilométrages cumulés et affichage chiffré du surcoût kilométrique imposé par la progression de niveau.

Extension 10 : Dynamique Météorologique et Modificateurs d'Arène (Climats)

— **Problématique décisionnelle :** Modéliser l'impact de conditions environnementales globales (climats) qui altèrent simultanément les formules de dégâts, appliquent des dégâts d'érosion passifs et favorisent certains types élémentaires au détriment d'autres.
— **Spécifications logicielles :**

— Définir une énumération `Weather(Enum)` dans `src/pyarena/extensions/weather.py` : `CLEAR` (Météo claire / neutre), `RAIN` (Pluie battante), `SUN` (Soleil zénithal), `SANDSTORM` (Tempête de sable).
— L'environnement de l'arène est instancié avec un climat actif, doté d'une durée finie $T_{\text{climat}} = 5$ tours (après quoi il revient à `CLEAR`), ou permanent pour le match.
— **Règles métier des altérations climatiques :**

1. *Pluie battante (`RAIN`) :*
   — Majore les dégâts des attaques de type **Eau** de +50% (multiplicateur climatique ×1,5).
   — Réduit les dégâts des attaques de type **Feu** de −50% (multiplicateur climatique ×0,5).
2. *Soleil zénithal (`SUN`) :*
   — Majore les dégâts des attaques de type **Feu** de +50% (×1,5).
   — Réduit les dégâts des attaques de type **Eau** de −50% (×0,5).
3. *Tempête de sable (`SANDSTORM`) :*
   — En clôture de chaque tour, inflige une perte passive de $\lfloor \text{PV}_{\max}/16 \rfloor$ PV à toute créature qui n'appartient **pas** aux types immunisés (les types *Roche*, *Sol* et *Acier* sont immunisés contre l'érosion du sable).
   — Confère un bonus passif de +50% en Défense ($D \leftarrow \lfloor 1{,}5 \times D \rfloor$) aux combattants de type *Roche*.

— **Formule des dégâts ajustée :** Le multiplicateur climatique $W \in \{0{,}5, 1{,}0, 1{,}5\}$ s'insère directement dans la formule de frappe du socle commun :

$$\text{Dégâts} = \max\left(1, \left\lfloor \left(\frac{22 \times P \times \frac{A_{\text{att}}}{D_{\text{déf}}}}{50} + 2 \right) \times M_{\text{total}} \times W \times U \times K \right\rfloor \right)$$

— **Tests unitaires (`tests/test_extension.py`) :**

— `test_rain_damage_modifiers` : Vérifier qu'une frappe Eau voit ses dégâts multipliés par 1,5 et qu'une frappe Feu voit ses dégâts divisés par deux sous la pluie.
— `test_sandstorm_immunity` : Valider que les créatures de type Roche ou Sol ne perdent aucun PV en fin de tour sous la tempête de sable, contrairement aux autres types.

— `test_weather_expiration` : Contrôler que le climat repasse bien à `CLEAR` après 5 tours et que les multiplicateurs redeviennent strictement neutres ($W = 1{,}0$).

— **Composant Streamlit :**

— *Vue Macro :* Heatmap Plotly comparant l'évolution du winrate d'un panel de 5 duels types sous les 4 conditions météo possibles (montrant par exemple comment un type Eau bat un type Électrik si la pluie compense sa faiblesse).

— *Vue Micro :* Télémétrie avec fond d'arène ou badge visuel changeant selon le climat actif (icônes gouttes de pluie, soleil ardent, nuage de sable) et suivi des dégâts d'érosion tour par tour.

Politique de Valorisation et Points de Bonus

Un mécanisme de **points de bonus** récompense l'approfondissement technique et les initiatives remarquables :

— **Plafond strict :** Les bonus permettent d'obtenir jusqu'à **+2 points** sur la Note Technique de Code (la note globale restant bornée à 20/20).

— **Condition impérative d'éligibilité :** Les points de bonus ne sont examinés que si le socle commun et l'extension attribuée sont intégralement validés par la CI GitHub Actions.

— **Inventaire obligatoire :** Chaque fonctionnalité bonus doit être répertoriée dans la section `## Fonctionnalités Bonus Implémentées` du fichier `README.md`.

Barème indicatif des initiatives possibles :

1. **Implémentation d'un agent tactique Minimax (profondeur 2 tours) (+1 pt) :**
   Pour les extensions 2 ou 4, concevoir un algorithme arborescent évaluant l'arbre des répliques adverses possibles pour maximiser l'espérance de survie.
2. **Déploiement public sur le Cloud (+0,5 pt) :**
   Héberger l'application sur *Streamlit Community Cloud* avec URL publique documentée dès l'en-tête du README.
3. **Double type & 18 types pour les trinômes (+0,75 pt) :**
   Prise en compte volontaire par un groupe de 3 du modèle complet à 18 types et doubles types croisés.
4. **Intégration multimédia audio officielle (+0,5 pt) :**
   Lecture dynamique des cris audio officiels (`st.audio`) via l'API lors des entrées en arène et des K.-O.

Travail Collaboratif, Traçabilité Git et Intégrité

Standards Git Obligatoires

1. **Branche `main` protégée :** Interdiction stricte de pousser directement sur `main`.

2. **Workflow par branches et Pull Requests :** Tout travail transite par une branche dédiée (`feat/...`, `fix/...`) et fait l'objet d'une Pull Request formelle relue par un pair.
3. **Audit contractuel : `CONTRIBUTIONS.md` :**
   Chaque membre doit documenter précisément ses livrables et la liste des PRs dont il est l'auteur et le relecteur :

Membre	Identifiant GitHub	Modules Développés	PRs Soumises	PRs Relues (*Approvals*)
Étudiant 1	`@login1`	Core POO, Tests unitaires	#1, #4	#2, #5
Étudiant 2	`@login2`	Client API, Streamlit	#2, #5	#3
Étudiant 3	`@login3`	Simulation, Extension	#3, #6	#1, #4

Détection du Plagiat et Contrôle d'Activité

— **Régularité temporelle :** Les dépôts révélant une injection massive de code en fin de semestre sans commits incrémentaux réguliers seront pénalisés individuellement.
— **Analyse de similarité AST (`copydetect`) :** L'intégralité du code source rendu fera l'objet d'un contrôle de similarité structurelle automatisé.

Modalités d'Évaluation et Grilles de Notation

La note finale est une moyenne pondérée associant la qualité de l'ingénierie logicielle livrée et la maîtrise individuelle démontrée lors de la soutenance :

$$\text{Note Finale} = \frac{3 \times \text{Note Code} + 2 \times \text{Note Soutenance}}{5}$$

Afin de garantir une stricte équité et de neutraliser tout effet de passager clandestin, **la Note de Code et la Note de Soutenance intègrent chacune une composante individuelle déterminante**. Plus de 50% du barème global dépend directement de l'investissement propre de chaque étudiant.

Note 1 : Code, Architecture et Produit (/20 — Mixte, Coeff 3)

La note de code se décompose en un **Socle Produit Collectif (/12)** et une note d'**Investissement Technique Individuel (/8)**.

A. Socle Produit Collectif (/12 points — Note d'Équipe)

Évalue la conformité du livrable logiciel déposé sur la branche `main` le jour du gel :

Critère Collectif	Barème	Critères d'évaluation objectifs
Architecture POO & Core	**/ 3**	Abstraction stricte (`BaseFighter`), encapsulation (`@property`), exceptions personnalisées.
Packaging & CI GitHub Actions	**/ 2**	`pyproject.toml` fonctionnel, workflow de CI au vert (si rouge le jour du rendu : -2 pts).
Pipeline Données & Parallélisme	**/ 2**	Stratégie de cache disque opérationnelle, parallélisation Monte-Carlo effective.
Application Streamlit	**/ 3**	Réactivité de l'application, présence des modules exigés, visualisations interactives Plotly/Folium.
Conformité de l'Extension	**/ 2**	Respect strict du cahier des charges métier de l'extension attribuée.

B. Contribution Technique Individuelle (/8 points — Note Individuelle)

Évalue l'implication effective et vérifiable de chaque membre dans la construction du code source, mesurée par l'audit Git et la traçabilité des Pull Requests :

Critère Individuel	Barème	Modalités d'évaluation objectives
Volume et consistance des PRs	**/ 3**	L'étudiant est l'auteur d'au moins 2 Pull Requests substantielles (code métier ou interface) ayant été fusionnées sur `main`.
Prise en charge des tests unitaires	**/ 3**	L'étudiant est l'auteur de tests unitaires formels (`tests/`) validant les fonctionnalités qu'il a développées (vérifié via `git blame` / historique des commits).

Critère Individuel	Barème	Modalités d'évaluation objectives
Régularité et revues de code	**/ 2**	Commits incrémentaux réguliers tout au long du semestre (absence de « dump » massif en fin de projet) et participation formelle aux revues de code de ses pairs (*Approvals*).

Note 2 : Soutenance Orale et Maîtrise Individuelle (/20 — Mixte, Coeff 2)

L'épreuve orale dure **15 minutes par équipe** (8 minutes d'exposé et démo live, 5-7 minutes de questions techniques individuelles, 2 minutes de transition).

Composante	Critère	Barème	Éléments évalués concrètement
Collectif (6 pts)	**Démonstration logicielle live**	**/ 3**	Fluidité de la démonstration Streamlit sans plantage.
	Supports visuels & Synthèse	**/ 3**	Diapositives claires et professionnelles, schémas d'architecture soignés, respect strict des 8 minutes de présentation.
Individuel (14 pts)	**Intervention orale préparée**	**/ 4**	Aisance de l'étudiant lors de son temps de parole dédié, clarté du propos et rigueur du vocabulaire employé.
	Questions techniques individuelles	**/ 5**	Aptitude à justifier à chaud ses choix de code, à expliciter une méthode précise et à commenter ses tests unitaires.
	Adéquation avec `CONTRIBUTIONS.md`	**/ 5**	Concordance rigoureuse entre les responsabilités déclarées dans le tableau contractuel et la maîtrise technique démontrée.

Calendrier Prévisionnel et Jalons de Suivi

— **Fin Septembre (Séance 2) :** Constitution des équipes et attribution définitive de l'extension thématique sur le tableur partagé.
— **Mi-Octobre (Jalon 1) :** Initialisation du dépôt GitHub, activation de la protection de branche et pipeline CI validé.
— **Fin Octobre (Jalon 2) :** Cœur POO achevé (`core/models.py`, `core/engine.py`), exceptions maison et premiers tests au vert.
— **Mi-Novembre (Jalon 3) :** Client API avec cache disque opérationnel, moteur Monte-Carlo parallélisé fonctionnel.
— **Début Décembre (Jalon 4) :** Extension thématique testée et intégrée dans le dashboard Streamlit.
— **Mi-Décembre :** Gel définitif des dépôts (`main`) et soutenances orales de fin de semestre.